

NAME;S.Harshavardhan

Reg;192524106

Experiment 1: Water Jug Problem

Aim

To solve the **Water Jug Problem** using Artificial Intelligence search techniques.

Algorithm

1. Start with both jugs empty.
2. Fill one of the jugs.
3. Pour water from one jug to another.
4. Empty a jug whenever required.
5. Repeat the steps until the required amount of water is obtained.
6. Stop.

Pseudo Code

START

Initialize Jug1 = 0, Jug2 = 0

WHILE goal is not reached

 IF Jug1 is empty

 Fill Jug1

 ELSE IF Jug2 is full

 Empty Jug2

 ELSE

 Pour water from Jug1 to Jug2

END WHILE

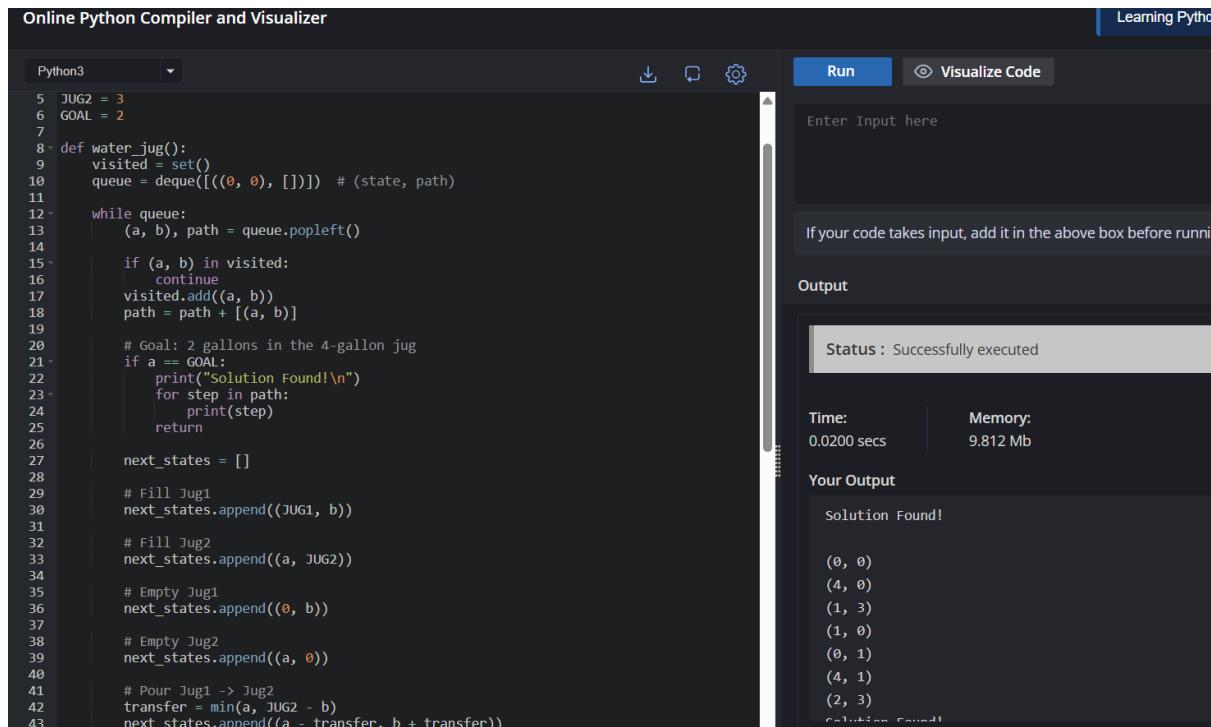
Display solution

STOP

Result

The Water Jug Problem was successfully solved, and the required amount of water was obtained using AI search techniques.

Outcome;



The screenshot displays an online Python compiler interface. On the left, a code editor shows a Python script for solving the Water Jug Problem using a breadth-first search algorithm. The script defines a function `water_jug()` that explores states of two jugs (JUG1 and JUG2) until the goal state (2 gallons in JUG1) is reached. The right panel shows the execution results, including a status message, execution time, memory usage, and the sequence of states visited.

```
Python3
5  JUG2 = 3
6  GOAL = 2
7
8  def water_jug():
9      visited = set()
10     queue = deque([(0, 0), []]) # (state, path)
11
12     while queue:
13         (a, b), path = queue.popleft()
14
15         if (a, b) in visited:
16             continue
17         visited.add((a, b))
18         path = path + [(a, b)]
19
20         # Goal: 2 gallons in the 4-gallon jug
21         if a == GOAL:
22             print("Solution Found!\n")
23             for step in path:
24                 print(step)
25             return
26
27     next_states = []
28
29     # Fill Jug1
30     next_states.append((JUG1, b))
31
32     # Fill Jug2
33     next_states.append((a, JUG2))
34
35     # Empty Jug1
36     next_states.append((0, b))
37
38     # Empty Jug2
39     next_states.append((a, 0))
40
41     # Pour Jug1 -> Jug2
42     transfer = min(a, JUG2 - b)
43     next_states.append((a - transfer, b + transfer))
```

Run Visualize Code

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 9.812 Mb

Your Output

Solution Found!

(0, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)
Solution Found!

Experiment 2: Cryptarithmic Problem

Aim

To solve the **Cryptarithmic Problem** using Constraint Satisfaction techniques.

Algorithm

1. Assign a unique digit to each letter.
2. Ensure no two letters have the same digit.
3. Do not assign zero to the leading letters.
4. Verify the arithmetic equation.
5. If the equation is not satisfied, assign different digits.
6. Repeat until a valid solution is found.
7. Stop.

Pseudo Code

START

Assign unique digits to each letter

REPEAT

Check all constraints

IF equation is satisfied

Display solution

EXIT

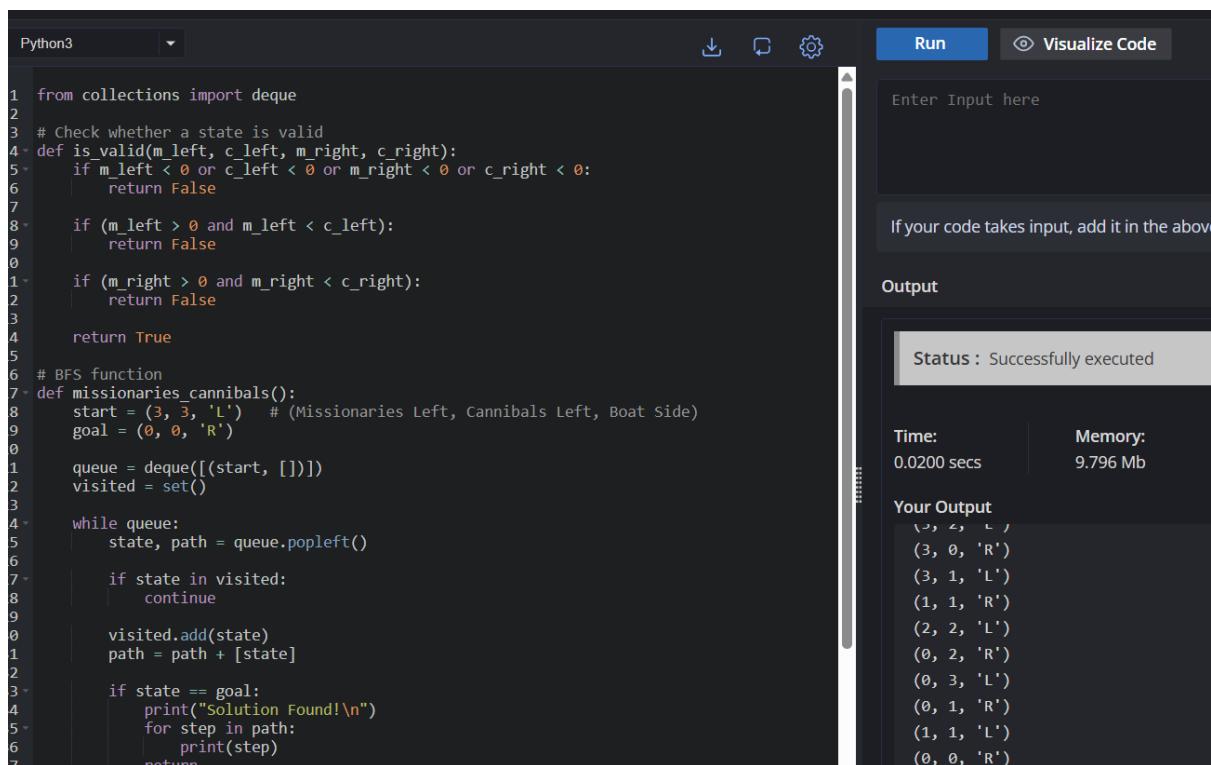
ELSE

Assign different digits

UNTIL solution is found

STOP

Outcome:



```
Python3
1 from collections import deque
2
3 # Check whether a state is valid
4 def is_valid(m_left, c_left, m_right, c_right):
5     if m_left < 0 or c_left < 0 or m_right < 0 or c_right < 0:
6         return False
7
8     if (m_left > 0 and m_left < c_left):
9         return False
10
11     if (m_right > 0 and m_right < c_right):
12         return False
13
14     return True
15
16 # BFS function
17 def missionaries_cannibals():
18     start = (3, 3, 'L') # (Missionaries Left, Cannibals Left, Boat Side)
19     goal = (0, 0, 'R')
20
21     queue = deque([(start, [])])
22     visited = set()
23
24     while queue:
25         state, path = queue.popleft()
26
27         if state in visited:
28             continue
29
30         visited.add(state)
31         path = path + [state]
32
33         if state == goal:
34             print("Solution Found!\n")
35             for step in path:
36                 print(step)
37             return
38
39     return None
```

Run Visualize Code

Enter Input here

If your code takes input, add it in the above

Output

Status : Successfully executed

Time:	Memory:
0.0200 secs	9.796 Mb

Your Output

```
(3, 3, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')
```

Result

The Cryptarithmic Problem was successfully solved by assigning unique digits to letters while satisfying all given constraints.